When you need to raise an exception, you can use one of the built-in Exception classes, or you can create one of your own. Your own exception classes should be subclasses of StandardError or one of its children, for the same reason we just gave. Making your exceptions children of StandardError ensures that regular Ruby processes will capture them appropriately.

Often, the only new piece of data associated with a custom exception is that it's a custom exception, so you can declare it in one line:

```
class MissingUserError < StandardError; end
```

Semicolons, which are rare in Ruby, are used to separate expressions when you put more than one on a line, this syntax is often used to indicate a class with no particular new data other than its parent class. By convention, custom exception class names end with Error.

Every Exception has associated with it a message string and a stack backtrace. If you define your own exceptions, you can add extra information, see Adding Information to Exceptions, on page 176.

## Handling Exceptions

Here's some simple code that uses the open-uri library to download the contents of a web page and write it to a file, line by line:

tut_exceptions/fetch_web_page/fetch1.rb

```
require "open-uri"
URI.open("https://pragprog.com/news/index.html") do |web_page|
  output = File.open("index.html", "w")
  while (line = web_page.gets)
    output.puts line
  end
  output.close
end
```

What happens if we get a fatal error halfway through? We certainly don't want to store an incomplete page to the output file.

Let's add some exception-handling code and see how it helps. To start exception handling, we enclose the code that could raise an exception in a begin/end block and use one or more rescue clauses to tell Ruby the types of exceptions we want to handle. If our code was already inside a method or an existing Ruby block, we would not need a separate begin/end block to trigger exception handling—the method or block is considered to be a begin/end block on its own. This code isn't in a method, so we need to create an explicit begin/ end block.

tut_exceptions/fetch_web_page/fetch2.rb

```
require "open-uri"

file_name = "index.html"
URI.open("https://pragprog.com/news/#{file_name}") do |web_page|
  output = File.open(file_name, "w")
  begin
    while (line = web_page.gets)
      output.puts line
    end
    output.close
  rescue StandardError
    $stderr.warn "Failed to download #{file_name}: #{$!}"
```